🔵 PART 1 — INFIX EVALUATION USING ONE STACK

✅ Basic Idea

We push:

• numbers

• operators

• '('

into ONE stack.

When we see):

We pop elements until (

Solve the expression inside bracket

Push result back to stack

Expression ends when all brackets are solved.

📌 Example 1

Expression:

(5 + 3)

Step 1

Stack = []

Read (

Push

Stack:

[(]

Read 5

Push

[(, 5]

Read +

Push

[(, 5, +]

Read 3

Push

[(, 5, +, 3]

Read)

Now we solve.

Pop 3

Pop +

Pop 5

5 + 3 = 8

Pop '('

Push result 8

Final Stack:

[8]

✅ Answer = 8

📌 Example 2

Expression:

(2 * (3 + 4))

Step by step:

Stack = []

Read (

[(]

Read 2

[(, 2]

Read *

[(, 2, *]

Read (

[(, 2, *, (]

Read 3

[(, 2, *, (, 3]

Read +

[(, 2, *, (, 3, +]

Read 4

[(, 2, *, (, 3, +, 4]

Read)

Now solve inside bracket:

Pop 4

Pop +

Pop 3

3 + 4 = 7

Pop '('

Push 7

Stack:

[(, 2, *, 7]

Read)

Now solve outer bracket:

Pop 7

Pop *

Pop 2

2 * 7 = 14

Pop '('

Push 14

Final Stack:

[14]

✅ Answer = 14

📌 Example 3

((1 + 2) * (3 + 4))

Solve inner brackets first:

(1 + 2) = 3

(3 + 4) = 7

Then:

3 * 7 = 21

Final Stack:

[21]

📌 Example 4

(8 / (4 - 2))

(4 - 2) = 2

8 / 2 = 4

Final Stack:

[4]

📌 Example 5

((5 + 5) - (3 * 2))

(5 + 5) = 10

(3 * 2) = 6

10 - 6 = 4

Final Stack:

[4]

🔵 HOW THIS METHOD WORKS

Very Important Rule:

When) comes:

Pop:

operand2

operator

operand1

Compute:

operand1 operator operand2

Push result back.

🔵 PART 2 — INFIX TO POSTFIX (ONE STACK)

Here we use:

• One stack for operators

• Output string for result

Operands directly go to output.

📌 Example 1

Expression:

(A + B)

Stack = []

Output = ""

Read (→ push

Stack: [(]

Read A → output

Output: A

Read + → push

Stack: [(, +]

Read B → output

Output: A B

Read)

Pop + → output

Pop (

Final Output:

A B +

📌 Example 2

(A + B) * C

Result:

A B + C *

📌 Example 3

(A + (B * C))

Result:

A B C * +

📌 Example 4

((A + B) - (C * D))

Result:

A B + C D * -

📌 Example 5

(A * B) + (C / D)

Result:

A B * C D / +

🔵 PART 3 — POSTFIX EVALUATION (ONE STACK)

Rule:

If operand → push

If operator →

pop operand1

pop operand2

apply

push result

📌 Example 1

5 3 +

Stack:

[5]

[5,3]

Read +

Pop 3

Pop 5

5 + 3 = 8

Push 8

Final:

[8]

📌 Example 2

2 3 4 + *

3 + 4 = 7

2 * 7 = 14

Final:

[14]

📌 Example 3

5 1 2 + 4 * + 3 -

1 + 2 = 3

3 * 4 = 12

5 + 12 = 17

17 - 3 = 14

Final:

[14]

🔵 NOW YOUR EXACT ALGORITHM (CLEAN FORMAT)

🔹 INFIX (ONE STACK METHOD)

Create an empty stack (S)

while (!end of expression)

```
{
   Get next input symbol (t)

   If (t != ')')
      S.push(t)

   else
   {
      S.pop(temp)

      while (temp != '(')
      {
         Store temp
         S.pop(temp)
      }

      Apply the operator on operand
      and push the result on stack
   }
}
```

- POSTFIX EVALUATION

```
Create an empty stack of operands

while (!end of expression)
{
   Get next input symbol (t)
```

```
    If (t is an operand)
        S.push(t)

    else
    {
        Apply the operation
        push result on stack
    }
}
```

Answer will be on top of stack

🔴 FINAL UNDERSTANDING

✓ Infix (your sir method) → One stack

✓ Solve when) comes

✓ Postfix → One stack

✓ Always remember operand order